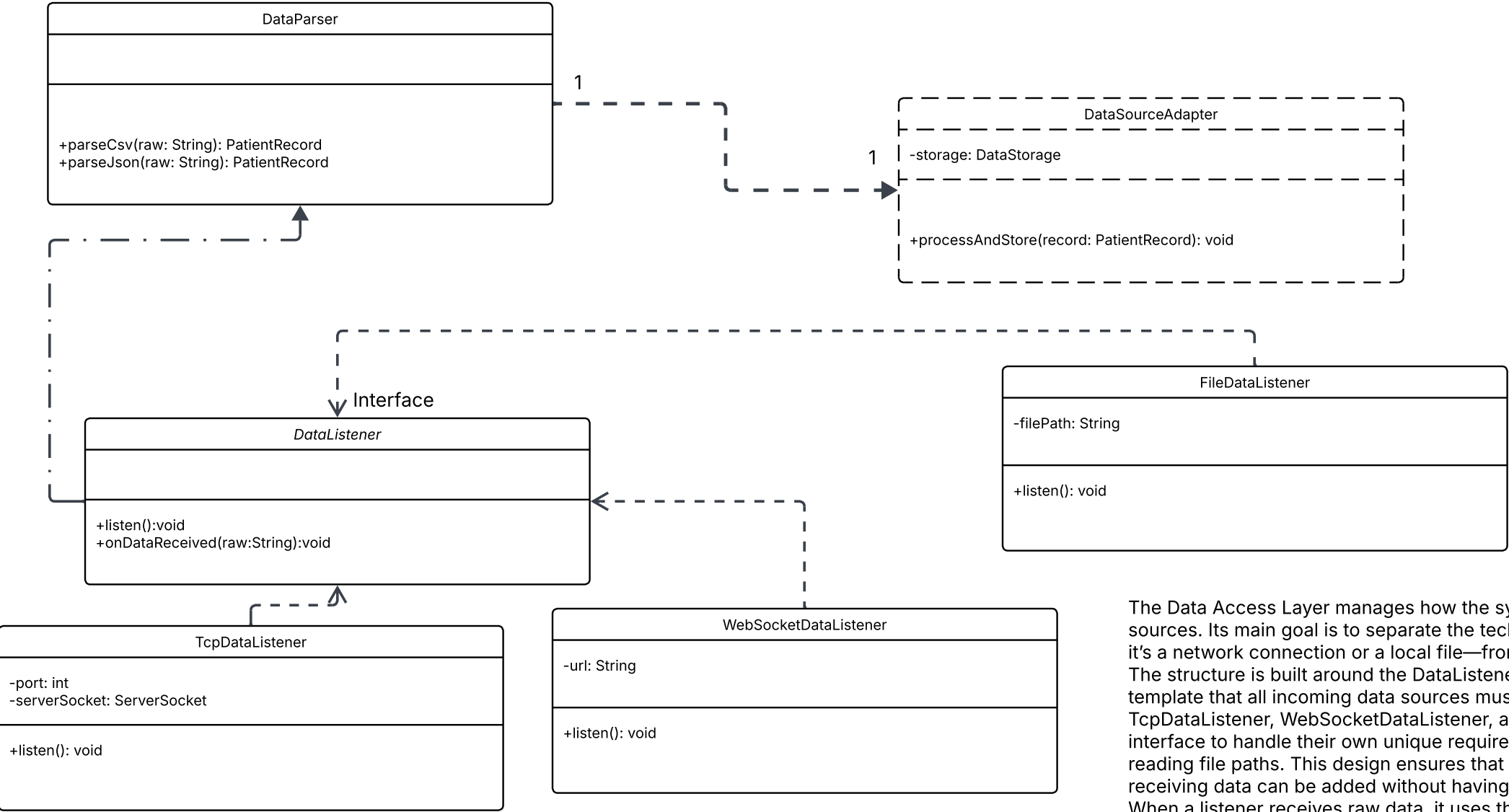




The Data Access Layer manages how the system receives information from external sources. Its main goal is to separate the technical details of the data source—whether it's a network connection or a local file—from the internal logic of the application. The structure is built around the **DataListener** interface. This serves as a common template that all incoming data sources must follow. The specific classes, such as **TcpDataListener**, **WebSocketDataListener**, and **FileDataListener**, implement this interface to handle their own unique requirements, like opening network ports or reading file paths. This design ensures that the system is flexible; new ways of receiving data can be added without having to rewrite the core parts of the software. When a listener receives raw data, it uses the **DataParser** to translate the information. The parser contains the logic needed to turn messy text formats like JSON or CSV into organized objects that the system understands. Once the data is cleaned and standardized, it is passed to the **DataSourceAdapter**. This class acts as the final bridge, taking the finished records and handing them over to the storage system. This setup keeps the rest of the application unaware of where the data came from, ensuring a clean and modular workflow.